

Laboratório 1 – Primeiro Programa em C

Duração: 2 horas | Trabalho individual | Entrega: arquivos .c ao final da aula

Objetivos deste laboratório

Ao final deste laboratório, a dupla deve ser capaz de:

- 1. escrever, compilar e executar um programa C simples no ambiente escolhido;
- 2. identificar e corrigir erros de compilação lendo as mensagens do compilador;
- 3. descrever o que cada parte de um programa mínimo em C faz;
- 4. distinguir os três tipos de erro – sintaxe, execução e lógica – a partir de exemplos concretos.

Atividade 1 – Anatomia do programa mínimo

20 min

O programa abaixo é o menor programa C válido que produz saída. Leia-o com atenção antes de digitá-lo.

```
#include <stdio.h>

int main() {
    printf("Ola, mundo!\n");
    return 0;
}
```

a) Salve o programa como ola.c, compile e execute. Registre o comando exato que você usou e a saída obtida:

b) Preencha a tabela relacionando cada elemento do programa ao seu papel:

Elemento	Papel no programa
#include <stdio.h>	
int main()	
{ }	
printf(...)	
"\n"	
return 0	

c) O que acontece se você remover o #include <stdio.h>? Remova, compile e anote a mensagem do compilador. Em seguida, restaure.

d) O que acontece se você remover o `return 0`? O programa ainda compila? Executa corretamente? O comportamento muda?

Atividade 2 – Os três tipos de erro

30 min

Nesta atividade você vai encontrar e corrigir exemplos dos três tipos de erro estudados na Aula 1: **sintaxe**, **execução** e **lógica**.

Bug – Erro de sintaxe

O programa abaixo não compila. Leia as mensagens do compilador, identifique **todos** os erros e corrija-os.

```
#include <stdio.h>

int main() {
    printf("Valor: %d\n" 42)
    printf("Fim\n");
    return 0
}
```

Quantos erros o compilador reportou? _____

Liste cada erro, a linha indicada e o que estava errado:

Observação: um único erro de sintaxe pode fazer o compilador reportar vários erros em cascata. Corrija o primeiro, recompile e veja se os demais desaparecem antes de tentar corrigi-los individualmente.

Bug – Erro de execução

O programa abaixo compila sem erros, mas falha ao executar para certas entradas. Descubra qual entrada causa a falha.

```
#include <stdio.h>

int main() {
    int a, b;
    printf("Digite dois inteiros: ");
    scanf("%d %d", &a, &b);
}
```

```
printf("Divisao: %d\n", a / b);  
return 0;  
}
```

a) Teste com $a = 10$, $b = 2$. Resultado: _____

b) Teste com $a = 10$, $b = 0$. O que acontece?

c) Corrija o programa para que ele imprima uma mensagem de erro adequada quando b for zero, em vez de travar. Escreva o código corrigido:

Bug – Erro de lógica

O programa abaixo compila e executa sem problemas, mas produz resultados incorretos. Encontre o erro.

```
#include <stdio.h>  
  
int main() {  
    int celsius;  
    float fahrenheit;  
    printf("Temperatura em Celsius: ");  
    scanf("%d", &celsius);  
    fahrenheit = celsius * 9 / 5 + 32;  
    printf("%.1f Fahrenheit\n", fahrenheit);  
    return 0;  
}
```

a) Teste com $celsius = 100$. Resultado esperado: 212.0. Resultado obtido: _____. O resultado está correto?

b) Teste com $celsius = 37$. Resultado esperado: 98.6. Resultado obtido: _____. O resultado está correto?

c) Identifique o erro. (Dica: releia a Aula 2 sobre divisão inteira.)

d) Corrija o programa e verifique com os dois casos de teste acima.

Antes de escrever qualquer programa desta atividade, escreva o algoritmo em português estruturado — uma lista de passos numerados descrevendo o que o programa deve fazer. Só então implemente em C.

Problema A — Área e perímetro de um retângulo

Escreva um programa que leia os lados (lado 1 e lado 2) de um retângulo e imprima a área e o perímetro com duas casas decimais.

Algoritmo em português:

Implemente, compile e teste com: lado 1 = 5, lado 2 = 3. Resultados esperados: área = 15.00, perímetro = 16.00.

Problema B — Troco

Escreva um programa que leia o preço de um produto (em reais, como número real) e o valor entregue pelo cliente, e calcule o troco. Se o valor entregue for menor que o preço, imprima uma mensagem de erro. *(Ainda sem usar if — apenas calcule e imprima o troco. O tratamento de erro será tema do Lab 3.)*

Algoritmo em português:

Implemente e teste com: preço = 7.50, valor entregue = 10.00. Resultado esperado: troco = 2.50.

Atividade 4 — Leitura e interpretação de código alheio

20 min

Leia o programa abaixo **sem executá-lo** e responda as perguntas.

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    printf("%d %d %d\n", n, n * n, n * n * n);
    return 0;
}
```

a) O que este programa calcula e imprime? Descreva em uma frase.

b) Para qual entrada o programa imprime 5 25 125? _____

c) Para $n = 10$, o que será impresso? _____

d) Para $n = 100$, o terceiro valor impresso é 1000000. Existe algum valor de n para o qual o terceiro valor seria incorreto? (Dica: qual é o valor máximo de um *int*?)

e) Compile e execute para verificar suas respostas. Havia alguma discrepância? Se sim, explique o motivo.

Desafio opcional

Calculadora de IMC

Escreva um programa que leia o peso (kg) e a altura (m) de uma pessoa e calcule o IMC (peso dividido pelo quadrado da altura). Exiba o resultado com duas casas decimais e uma faixa de classificação baseada apenas em comparações com constantes — sem usar *if* por enquanto, apenas calcule e exiba o valor.

Depois, responda: qual seria o problema em usar *int* em vez de *float* ou *double* para guardar o IMC? Escreva a resposta como comentário no código.